

ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Facultad de Ingeniería en Electricidad y Computación

Software Engineering II

ACCEPTANCE TEST WORKSHOP

Inventory Manager

Pruebas de aceptación con Gherkin y Behave (BDD)

INTEGRANTES

Gutierrez Macias Javier Alejandro

Severino Ponce Juan Argenis

Rodriguez Romero Luis Fernando

Quimi Poveda Michael Crescencio

Cevallos Vinces Nahin Jussephe

Andradez Veloz Mariu Isabeau

Profesor: Dr. Carlos Mera

Paralelo: 1 I Término 2026

Guayaquil, 6 de julio de 2026

Repositorio del proyecto

<https://github.com/SKEIILATT/AcceptanceTestingG1>

Índice

1. Introducción	3
2. Desarrollo.....	4
2.1. Parte 1: Configuración del proyecto	4
2.1.1. Requerimientos implementados	5
2.2. Parte 2: Desarrollo de pruebas	5
2.2.1. Estructura del proyecto.....	5
2.2.2. Feature files	5
2.2.3. Steps (Python)	7
2.2.4. Evidencia de ejecución	9
2.3. Parte 3: Proceso iterativo	9
2.4. Evidencia de participación del equipo	10
3. Conclusiones	12
4. Recomendaciones	13
5. Referencias.....	14

1. Introducción

El desarrollo dirigido por comportamiento (Behavior Driven Development, BDD) es una forma de trabajo que acerca a las personas del negocio y a las personas técnicas de un equipo de software. Lo logra fomentando la colaboración entre roles para construir un entendimiento compartido del problema, trabajando en iteraciones pequeñas y rápidas que aumentan la retroalimentación, y produciendo documentación del sistema que se verifica automáticamente contra su comportamiento real.

Las pruebas de aceptación (Acceptance Testing) son un nivel de prueba de software en el que se comprueba la aceptabilidad de un sistema con respecto a las necesidades del usuario, los requisitos y los procesos de negocio. Su propósito es determinar si el sistema satisface los criterios de aceptación y, con ello, decidir si es apto para ser entregado.

El objetivo de este workshop es validar el correcto funcionamiento de un sistema mediante pruebas de aceptación. Para escribir dichas pruebas se emplea el lenguaje Gherkin (del marco Cucumber) y, como motor de ejecución en Python, la librería Behave. El sistema sobre el cual se aplicó es Inventory Manager, una aplicación de línea de comandos que permite gestionar productos: agregarlos, listarlos, actualizar su cantidad, eliminarlos y —como funcionalidad adicional— buscarlos por nombre.

El código fuente completo del proyecto se encuentra en el siguiente repositorio: <https://github.com/SKEILATT/AcceptanceTestingG1>

2. Desarrollo

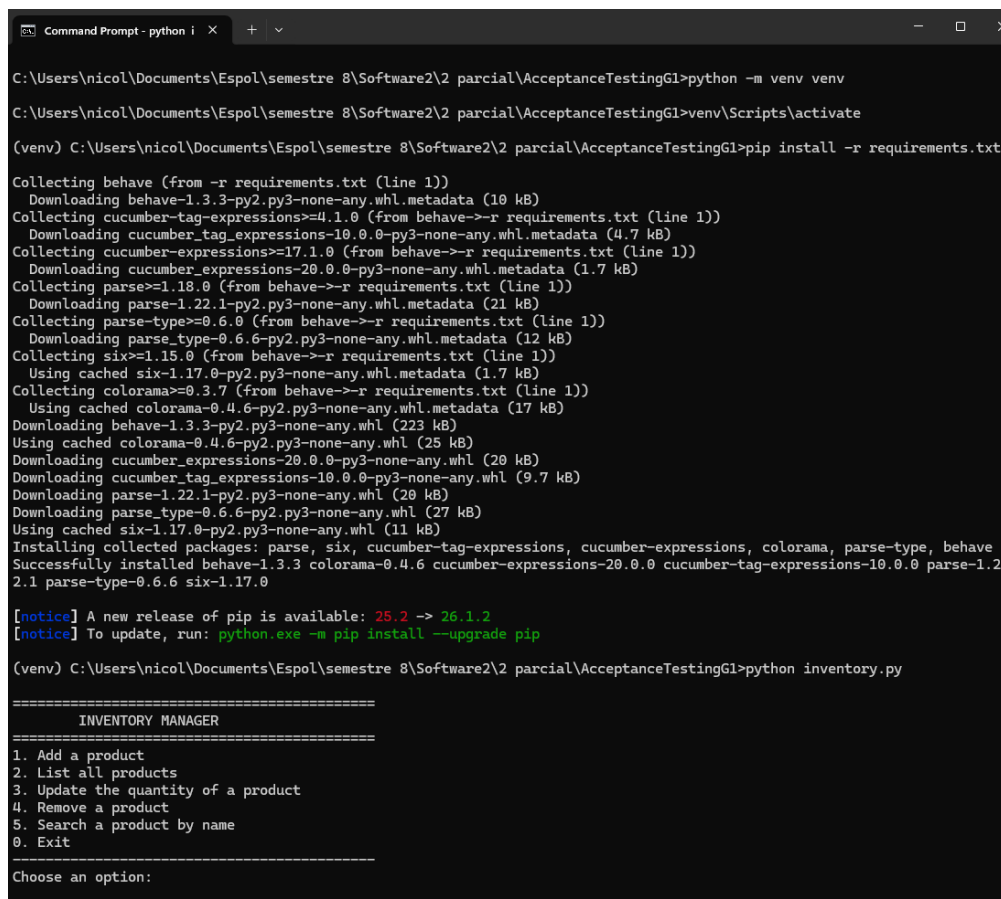
2.1. Parte 1: Configuración del proyecto

Se creó un repositorio en GitHub y, dentro del directorio del proyecto, un entorno virtual de Python que aísla las dependencias del taller. Con el entorno activo se instaló la librería Behave a partir del archivo requirements.txt. Los comandos utilizados fueron:

```
# Windows
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
pip3 install -r requirements.txt
```

Se verificó, además, que la aplicación principal se ejecuta sin errores mediante el comando python inventory.py, el cual despliega el menú interactivo del gestor de inventario. La Figura 1 evidencia la creación y activación del entorno virtual, la instalación exitosa de Behave y sus dependencias, y la ejecución correcta de la aplicación.



```
Command Prompt - python i x + v
C:\Users\nicol\Documents\Espol\semestre 8\Software2\2 parcial\AcceptanceTestingG1>python -m venv venv
C:\Users\nicol\Documents\Espol\semestre 8\Software2\2 parcial\AcceptanceTestingG1>venv\Scripts\activate
(venv) C:\Users\nicol\Documents\Espol\semestre 8\Software2\2 parcial\AcceptanceTestingG1>pip install -r requirements.txt
Collecting behave (from -r requirements.txt (line 1))
  Downloading behave-1.3.3-py2.py3-none-any.whl.metadata (10 kB)
Collecting cucumber-tag-expressions>=4.1.0 (from behave->-r requirements.txt (line 1))
  Downloading cucumber_tag_expressions-10.0.0-py3-none-any.whl.metadata (4.7 kB)
Collecting cucumber-expressions>=17.1.0 (from behave->-r requirements.txt (line 1))
  Downloading cucumber_expressions-20.0.0-py3-none-any.whl.metadata (1.7 kB)
Collecting parse>=1.18.0 (from behave->-r requirements.txt (line 1))
  Downloading parse-1.22.1-py2.py3-none-any.whl.metadata (21 kB)
Collecting parse-type>=0.6.0 (from behave->-r requirements.txt (line 1))
  Downloading parse_type-0.6.6-py2.py3-none-any.whl.metadata (12 kB)
Collecting six>=1.15.0 (from behave->-r requirements.txt (line 1))
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Collecting colorama>=0.3.7 (from behave->-r requirements.txt (line 1))
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Downloading behave-1.3.3-py2.py3-none-any.whl (223 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading cucumber_expressions-20.0.0-py3-none-any.whl (20 kB)
Downloading cucumber_tag_expressions-10.0.0-py3-none-any.whl (9.7 kB)
Downloading parse-1.22.1-py2.py3-none-any.whl (20 kB)
Downloading parse_type-0.6.6-py2.py3-none-any.whl (27 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: parse, six, cucumber-tag-expressions, cucumber-expressions, colorama, parse-type, behave
Successfully installed behave-1.3.3 colorama-0.4.6 cucumber-expressions-20.0.0 cucumber-tag-expressions-10.0.0 parse-1.22.1 parse-type-0.6.6 six-1.17.0

[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Users\nicol\Documents\Espol\semestre 8\Software2\2 parcial\AcceptanceTestingG1>python inventory.py

=====
INVENTORY MANAGER
=====
1. Add a product
2. List all products
3. Update the quantity of a product
4. Remove a product
5. Search a product by name
0. Exit
-----
Choose an option:
```

Figura 1. Evidencia de la configuración: creación del entorno virtual, instalación de Behave y ejecución de inventory.py.

2.1.1. Requerimientos implementados

Se implementaron los cuatro requerimientos sugeridos y se agregó un quinto requerimiento definido por el grupo. El producto se modeló con cuatro atributos (mínimo exigido): name, category, quantity y price.

- Agregar un producto al inventario.
- Listar todos los productos del inventario.
- Actualizar la cantidad de un producto.
- Eliminar un producto del inventario.
- **[Nuevo requerimiento agregado]:** Buscar un producto por su nombre. La aplicación informa si el producto existe en el inventario o, en caso contrario, devuelve un mensaje indicando que no fue encontrado. Este requerimiento habilita además una prueba de interacción fallida.

2.2. Parte 2: Desarrollo de pruebas

2.2.1. Estructura del proyecto

La lógica de negocio se separó de la interfaz de línea de comandos: la clase Inventory vive en inventory_core.py y es utilizada tanto por la aplicación como por las pruebas, de modo que se prueba exactamente el mismo código que se ejecuta en producción. Los steps se dividieron en un archivo por funcionalidad; Behave carga automáticamente todos los archivos .py de la carpeta steps/.

```
inventory-manager/  
  inventory.py           # Aplicación CLI (archivo principal)  
  inventory_core.py      # Lógica de negocio (Product, Inventory)  
  requirements.txt  
  README.md  
  features/  
    environment.py       # Hooks de Behave (sys.path)  
    add_product.feature  # Feature 1  
    list_products.feature # Feature 2  
    update_quantity.feature # Feature 3  
    remove_product.feature # Feature 4  
    search_product.feature # Feature 5 (añadida)  
  steps/  
    common_steps.py      # Given compartidos  
    add_steps.py  
    list_steps.py  
    search_steps.py  
    update_steps.py  
    remove_steps.py
```

2.2.2. Feature files

A continuación se presentan las 5 features con sus 5 escenarios escritos en Gherkin. Cada escenario usa las palabras clave Given (estado inicial conocido), When (acción del usuario) y Then (resultado observable).

```
# add_product.feature -- Feature 1  
Feature: Add a product to the inventory  
  
  Scenario: Add a product to the inventory  
    Given the inventory is empty  
    When the user adds a product "Coffee"  
    Then the inventory should contain "Coffee"
```

```
# list_products.feature -- Feature 2  
Feature: List all products in the inventory
```

```

Scenario: List all products in the inventory
  Given the inventory contains products:
    | Product |
    | Coffee |
    | Sugar  |
  When the user lists all products
  Then the output should contain:
    """
    Products:
    - Coffee
    - Sugar
    """

```

```

# update_quantity.feature -- Feature 3
Feature: Update the quantity of a product

Scenario: Update the quantity of a product
  Given the inventory contains products:
    | Product | Quantity |
    | Coffee | 10       |
  When the user updates product "Coffee" to quantity "25"
  Then the inventory should show product "Coffee" with quantity "25"

```

```

# remove_product.feature -- Feature 4
Feature: Remove a product from the inventory

Scenario: Remove a product from the inventory
  Given the inventory contains products:
    | Product |
    | Coffee |
    | Sugar  |
  When the user removes the product "Coffee"
  Then the inventory should not contain "Coffee"

```

```

# search_product.feature -- Feature 5 (añadida, interacción fallida)
Feature: Search a product by name

Scenario: Search for a product that does not exist
  Given the inventory contains products:
    | Product |
    | Sugar  |
  When the user searches for the product "Coffee"
  Then the output should be "Product Coffee was not found"

```

2.2.3. Steps (Python)

Cada línea Gherkin se implementó como un step en Python. Los datos se comparten a través del objeto context, que Behave reinicia por cada escenario para evitar que el estado se filtre de un escenario a otro. Los Given comunes se ubican en un único archivo para no duplicarlos.

```

# common_steps.py -- Given compartidos
from behave import given
from inventory_core import Inventory

@given('the inventory is empty')
def step_impl(context):
    context.inventory = Inventory()

@given('the inventory contains products:')
def step_impl(context):
    context.inventory = Inventory()
    for row in context.table:

```

```

        name = row['Product']
        if 'Quantity' in row.headings:
            context.inventory.add_product(name, quantity=int(row['Quantity']))
        else:
            context.inventory.add_product(name)

```

```

# add_steps.py
from behave import when, then

@when('the user adds a product "{product}"')
def step_impl(context, product):
    context.output = context.inventory.add_product(product)

@then('the inventory should contain "{product}"')
def step_impl(context, product):
    assert context.inventory.contains(product), \
        f'Product "{product}" not found in the inventory'

```

```

# list_steps.py
@when('the user lists all products')
def step_impl(context):
    context.output = context.inventory.list_products()

@then('the output should contain:')
def step_impl(context):
    expected = context.text.strip()
    assert expected in context.output, \
        f'Expected to find:\n{expected}\nbut got:\n{context.output}'

```

```

# update_steps.py
@when('the user updates product "{product}" to quantity "{quantity}"')
def step_impl(context, product, quantity):
    context.output = context.inventory.update_quantity(product, int(quantity))

@then('the inventory should show product "{product}" with quantity "{quantity}"')
def step_impl(context, product, quantity):
    product_obj = context.inventory.get_product(product)
    assert product_obj is not None, f'Product "{product}" not found'
    assert product_obj.quantity == int(quantity), \
        f'Expected quantity {quantity} but got {product_obj.quantity}'

```

```

# remove_steps.py
@when('the user removes the product "{product}"')
def step_impl(context, product):
    context.output = context.inventory.remove_product(product)

@then('the inventory should not contain "{product}"')
def step_impl(context, product):
    assert not context.inventory.contains(product), \
        f'Product "{product}" is still in the inventory'

```

```

# search_steps.py
@when('the user searches for the product "{product}"')
def step_impl(context, product):
    context.output = context.inventory.search_product(product)

@then('the output should be "{message}"')
def step_impl(context, message):
    assert context.output == message, \
        f'Expected "{message}" but got "{context.output}"'

```

La clase Inventory (en inventory_core.py) concentra la lógica invocada por los steps:

```
# inventory_core.py (resumen)
class Product:
    def __init__(self, name, category="General", quantity=0, price=0.0):
        self.name = name
        self.category = category
        self.quantity = int(quantity)
        self.price = float(price)

class Inventory:
    def __init__(self):
        self._products = {}

    def contains(self, name):
        return name in self._products

    def get_product(self, name):
        return self._products.get(name)

    def add_product(self, name, category="General", quantity=0, price=0.0):
        if name in self._products:
            return f"Product {name} already exists"
        self._products[name] = Product(name, category, quantity, price)
        return f"Product {name} was added"

    def list_products(self):
        if not self._products:
            return "Products:\n(no products)"
        lines = ["Products:"] + [f"- {n}" for n in self._products]
        return "\n".join(lines)

    def update_quantity(self, name, quantity):
        if name not in self._products:
            return f"Product {name} was not found"
        self._products[name].quantity = int(quantity)
        return f"Product {name} updated to quantity {quantity}"

    def remove_product(self, name):
        if name in self._products:
            del self._products[name]
            return f"Product {name} was removed"
        return f"Product {name} was not found"

    def search_product(self, name):
        if name in self._products:
            return f"Product {name} found"
        return f"Product {name} was not found"
```

2.2.4. Evidencia de ejecución

Al ejecutar el comando behave, las cinco pruebas de aceptación se ejecutaron satisfactoriamente. La Figura 2 muestra la estructura del proyecto en Visual Studio Code junto con la salida completa de Behave, donde todos los escenarios aparecen en verde.

```
5 features passed, 0 failed, 0 skipped
5 scenarios passed, 0 failed, 0 skipped
15 steps passed, 0 failed, 0 skipped
Took 0min 0.002s
```

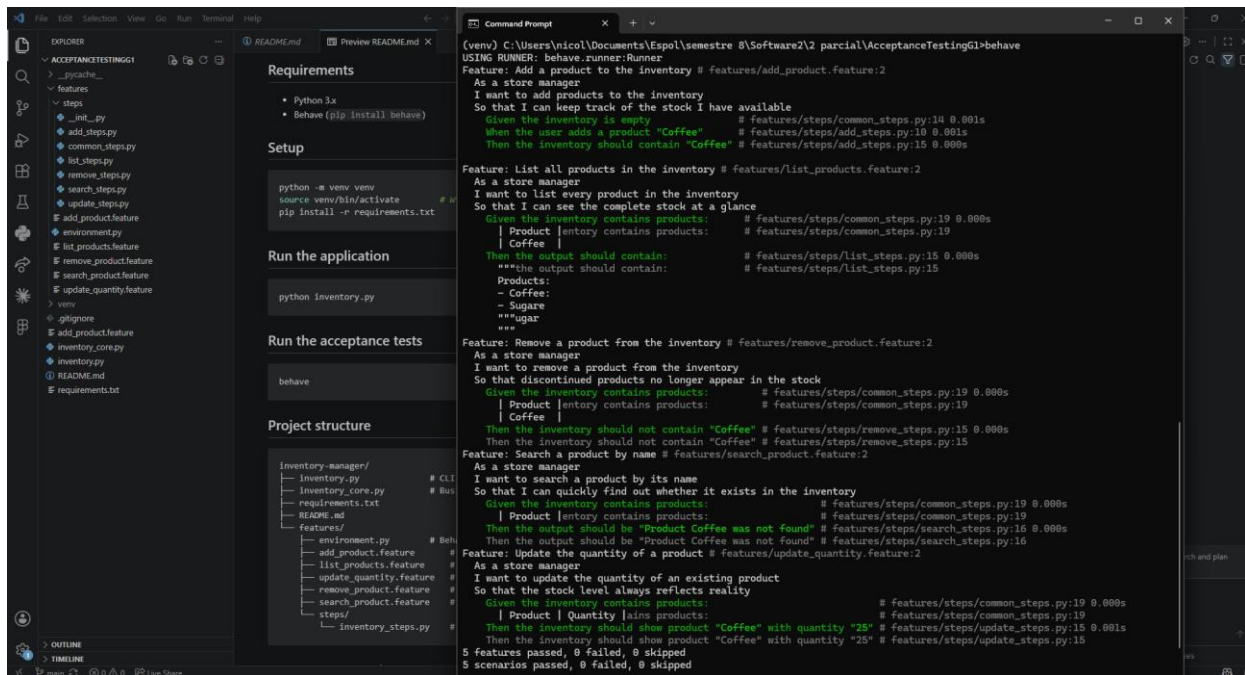



Figura 2. Estructura del proyecto en VS Code y ejecución exitosa de behave: 5 features, 5 escenarios y 15 steps aprobados.

2.3. Parte 3: Proceso iterativo

Para revisar los resultados y comprobar la utilidad de las pruebas, se introdujo deliberadamente un defecto en la lógica de actualización de cantidad dentro de `inventory_core.py`: en el método `update_quantity` se sumó una unidad de más al valor indicado (`quantity + 1`). Al ejecutar nuevamente behave, la suite detectó la regresión de inmediato y reportó el fallo con el detalle exacto de la aserción, tal como se evidencia en la Figura 3.

```
Scenario: Update the quantity of a product
...
    Then the inventory should show product "Coffee" with quantity "25"
    ASSERT FAILED: Expected quantity 25 but got 26

Failing scenarios:
  features/update_quantity.feature:7 Update the quantity of a product

4 features passed, 1 failed, 0 skipped
4 escenarios passed, 1 failed, 0 skipped
14 steps passed, 1 failed, 0 skipped
```

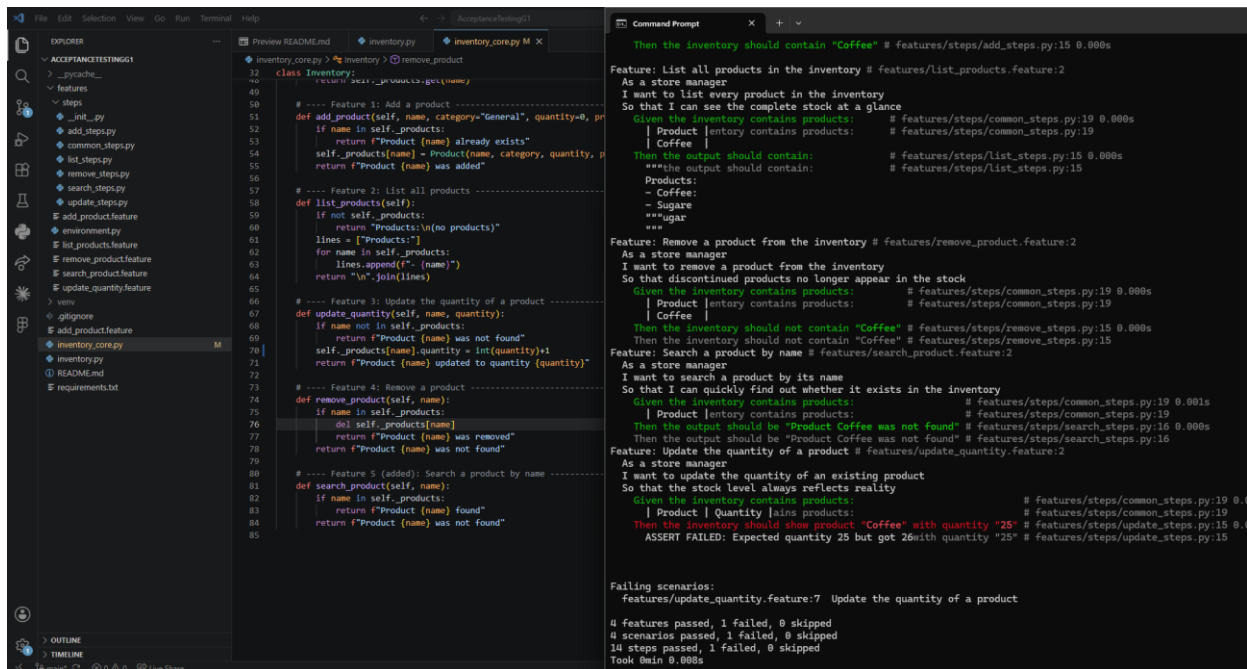


Figura 3. Defecto introducido en `update_quantity` (línea con `quantity + 1`) y detección de la regresión por la suite de Behave: `ASSERT FAILED`, 1 escenario fallido.

Tras corregir el error en `inventory_core.py` (asignando exactamente la cantidad indicada, sin el incremento adicional), se volvió a ejecutar la suite completa y todos los escenarios pasaron nuevamente, como se mostró en la Figura 2. Con esto se evidencia el valor del enfoque BDD: las pruebas de aceptación actúan como una red de seguridad frente a errores introducidos durante el mantenimiento. En la versión final del código no quedan errores conocidos.

2.4. Evidencia de participación del equipo

El desarrollo del proyecto se realizó de forma colaborativa a través del repositorio en GitHub. La Figura 4 muestra la sección Contributors del repositorio, donde se evidencian los commits realizados por los integrantes del grupo durante el desarrollo del taller.

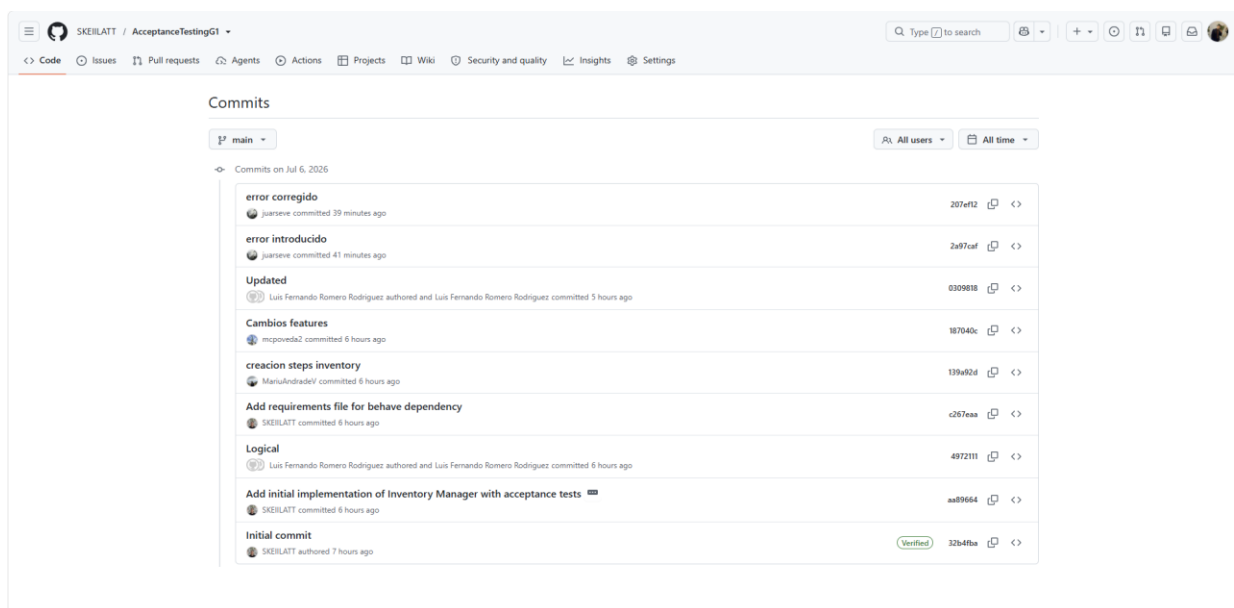
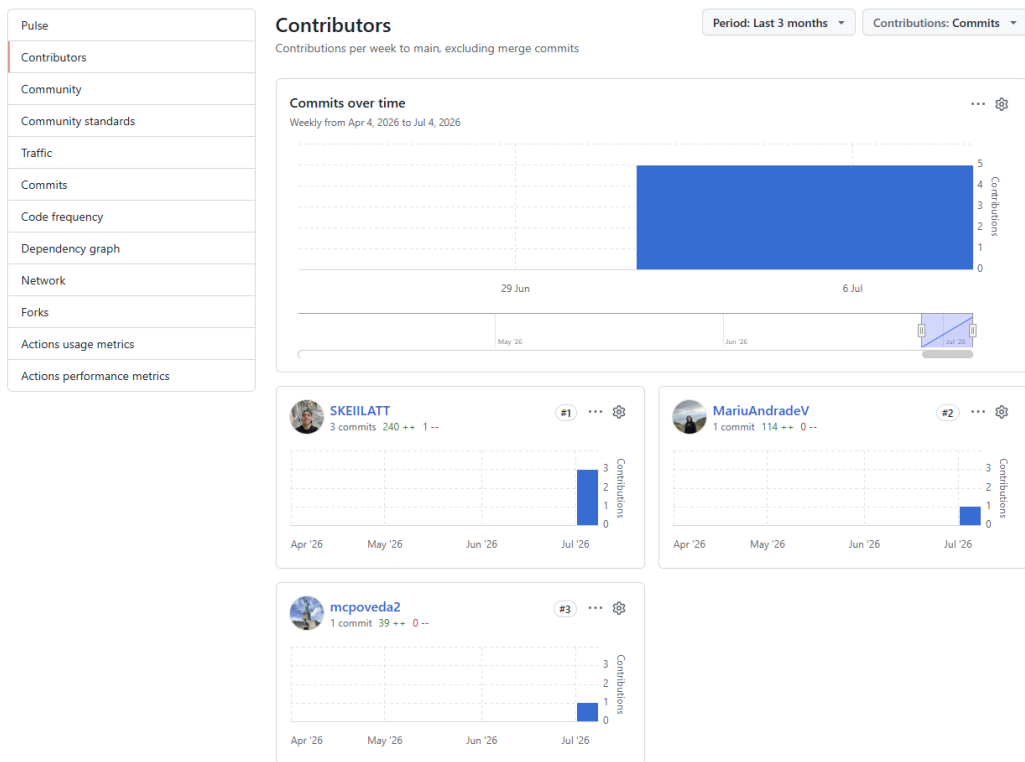


Figura 4 y 5. Evidencia de participación: contribuciones (commits) de los integrantes del grupo en el repositorio de GitHub.

3. Conclusiones

- Se validó el correcto funcionamiento del sistema Inventory Manager mediante cinco pruebas de aceptación automatizadas, cumpliendo el objetivo del workshop.
- El enfoque BDD con Gherkin permitió redactar especificaciones legibles tanto por perfiles de negocio como técnicos, que además son ejecutables y funcionan como documentación viva del sistema.
- Separar la lógica de negocio de la interfaz permitió que las pruebas ejerciten el mismo código que utiliza la aplicación, aumentando su fiabilidad; y el proceso iterativo confirmó que la suite detecta regresiones de forma inmediata.

4. Recomendaciones

- Ejecutar la suite de Behave ante cada cambio, idealmente dentro de un flujo de integración continua, para detectar regresiones de forma temprana.
- Utilizar el objeto context en lugar de variables globales para preservar el aislamiento entre escenarios.
- Ampliar la cobertura con más escenarios de interacción fallida (por ejemplo, actualizar un producto inexistente o rechazar cantidades negativas) y emplear Scenario Outline con Examples para parametrizar casos similares y reducir la duplicación.

5. Referencias

- [1] Software Testing Fundamentals, “Acceptance Testing”. [Online]. Available: <http://softwaretestingfundamentals.com/acceptance-testing/>
- [2] R. Hewitt, “Gherkin for Business Analysts”, Modern Analyst, 2017. [Online]. Available: <https://www.modernanalyst.com/Resources/Articles/tabid/115/ID/3810/Gherkin-for-Business-Analysts.aspx>
- [3] “Gherkin Reference”. [Online]. Available: <https://cucumber.io/docs/gherkin/reference/>
- [4] Behave, “Read the Docs”. [Online]. Available: <https://behave.readthedocs.io/en/latest/tutorial/#feature-files>
- [5] Behave API Reference. [Online]. Available: <https://behave.readthedocs.io/en/latest/api/>
- [6] Repositorio del proyecto Inventory Manager (GitHub). [Online]. Available: <https://github.com/SKEILATT/AcceptanceTestingG1>